RAG System Documentation

Rodent Knowledge Base

Built with: Qwen2-1.5B | FAISS | BGE Embeddings | Gradio

Platform: Google Colab Free Tier T4 GPU

1. Introduction

This project implements a RAG (Retrieval Augmented Generation) system that enables users to ask questions about rodents and receive accurate answers grounded in Wikipedia documents, with source citations and a confidence indicator. The entire system runs in Google Colab using free-tier GPU resources.

Documents Used (13 Wikipedia Articles)

- Rodent (overview)
- List of rodents (overview)
- Beaver
- Nutria
- Gopher
- Chinchilla
- Squirrel
- Capybara
- Guinea pig
- Marmot
- Pika
- Rabbit
- Rat

2. System Architecture

The pipeline follows these sequential steps for every user query:

- 1. User Question - text input from the web interface
- 2. Query Transformation (optional) - rewrite, expand, or HyDE using Qwen2
- 3. Embedding - BAAI/bge-small-en-v1.5 converts query to 384-dim vector
- 4. Retrieval - FAISS dense search + BM25 sparse search fused as hybrid
- 5. Reranking (optional) - cross-encoder ms-marco-MiniLM-L-6-v2
- 6. Confidence Computation - based on retrieval similarity scores
- 7. Prompt Engineering - grounded prompt with strict rules against hallucination
- 8. LLM Generation - Qwen2-1.5B-Instruct with 4-bit NF4 quantization
- 9. Response - Answer + Source Citations + Confidence Score (High/Medium/Low)

3. Technology Choices & Justifications

3.1 LLM - Qwen2-1.5B-Instruct

Model	Params	Disk	Quality	Colab
Phi-3-mini	3.8B	~8GB	Excellent	Borderline
Qwen2-1.5B	1.5B	~3GB	Good	Yes
Gemma-2B	2B	~5GB	Very Good	Yes
Phi-2	2.7B	~6GB	Very Good	Yes
TinyLlama	1.1B	~2GB	Decent	Yes

Qwen2-1.5B-Instruct was selected because it fits in Colab free tier GPU memory with 4-bit NF4 quantization (only 1-2GB), consistently outperforms TinyLlama on instruction following benchmarks, and reliably follows the grounding prompt to avoid hallucination. 4-bit quantization reduces memory footprint by 75% with only 1-3% quality degradation.

3.2 Hugging Face Transformers vs Ollama

Hugging Face Transformers was chosen over Ollama because Ollama is designed for local environments and expects to manage its own server process, causing issues in Colab. Transformers provides direct CUDA access, seamless 4-bit quantization via bitsandbytes, and all model families are available directly from Hugging Face Hub.

3.3 Vector Store - FAISS vs ChromaDB vs Qdrant

FAISS was chosen as primary vector store. ChromaDB was implemented for comparison. Qdrant was rejected because it requires a running Docker container or cloud instance. ChromaDB in Colab had version conflicts with NumPy 2.0 making FAISS the more reliable choice.

Feature	FAISS	ChromaDB
Type	In-memory	Persistent
Speed	Very fast 0.011s	Fast 0.014s
Metadata filtering	Manual (post)	Native (where clause)
Persistence	Manual save/load	Automatic
Session restart	Reload needed	Auto reload

4. Phase 2 - Chunking Strategy

4.1 Fixed Size Chunking

Splits text at a fixed character count with overlap between chunks. Simple and fast but ignores meaning completely. Can cut mid-sentence or mid-topic.

Chunk Size	Total Chunks	Avg Length
256 chars	1472	252 chars
512 chars	658	506 chars
1000 chars	324	979 chars

4.2 Recursive Character Splitting

Tries to split at natural language boundaries in priority order: paragraphs > line breaks > sentences > words > characters. Respects natural text boundaries and produces more coherent chunks than fixed size. Still based on character counts but does not understand meaning.

Chunk Size	Total Chunks	Avg Length
256 chars	1657	shorter
512 chars	705	balanced
1000 chars	333	longer

4.3 Semantic Chunking

Splits based on meaning by embedding every sentence and splitting where cosine similarity between consecutive sentences drops below a threshold. Produces the most meaningful chunks where each chunk stays on one topic. Much slower than other methods as it requires embedding every sentence.

Threshold	Total Chunks	Avg Length
0.2	469	643 chars
0.3	632	477 chars
0.5	1164	258 chars

4.4 Comparison and Winner

Method	Total Chunks	Avg Length	Min	Max
Fixed Size	677	493	83	500
Recursive	729	418	66	500
Semantic	632	477	21	1603

WINNER: Recursive chunking (512 chars, 50 overlap) was chosen for the main pipeline. It produces coherent chunks that respect natural text boundaries while being fast and predictable. Semantic chunking produces higher quality but is significantly slower.

5. Phase 3 - Embedding Model Benchmarking

5.1 Models Evaluated

Model	Size	Speed	Quality	Memory
all-MiniLM-L6-v2	22MB	Very Fast	Good	~90MB
BAAI/bge-small-en-v1.5	33MB	Fast	Very Good	~130MB
BAAI/bge-base-en-v1.5	109MB	Medium	Excellent	~440MB
thenlper/gte-small	67MB	Fast	Very Good	~250MB
intfloat/e5-small-v2	33MB	Fast	Very Good	~130MB

5.2 Benchmark Results on Real Chunks

Query	MiniLM Score	BGE Score
What is the largest rodent	0.711	0.758
How do beavers build dams	0.415	0.660
Where do chinchillas come from	0.368	0.625

BGE-small-en-v1.5 is the clear winner. It produces significantly higher confidence scores (BGE 0.660 vs MiniLM 0.415 for beaver query), more consistent rankings across top results, and handles specific queries much better (chinchilla: BGE 0.625 vs MiniLM 0.368). Both models agree on top results but BGE does it with much higher certainty. DECISION: BAAI/bge-small-en-v1.5 is used for all embeddings in the main pipeline.

6. Phase 6 - Baseline RAG Pipeline

6.1 What is RAG?

RAG (Retrieval Augmented Generation) combines a retrieval system with a language model to produce answers grounded in specific documents rather than relying on model training data. When a user asks a question, the most relevant chunks are retrieved from the vector store, inserted into a carefully engineered prompt, and the LLM generates an answer based only on that context. This prevents hallucination and enables source citations.

6.2 Prompt Engineering

The prompt was carefully engineered to keep the small LLM grounded. Key decisions:

- Question placed at END after context (small models pay more attention to recent text)
- Exact refusal phrase specified: "I cannot find this information in the available documents"
- Sources numbered for easy citation by the model
- Low temperature (0.1) for focused deterministic answers
- Explicit rule: "Do NOT use any outside knowledge"

6.3 Confidence Indicator

Confidence is computed from retrieval similarity scores:

Level	Threshold	Action
High	score >= 0.75	Answer generated
Medium	score >= 0.50	Answer generated
Low	score < 0.50	Refusal returned without LLM call

Formula: confidence = (top_score x 0.7) + (avg_score x 0.3)

7. Phase 7 - Advanced Retrieval Techniques

7.1 Hybrid Search (BM25 + Dense)

Dense retrieval understands meaning but may miss exact keyword matches. BM25 finds exact keyword matches but misses semantic similarity. Combining both with score fusion gives the best of both worlds. Formula: Final score = (0.4 x BM25) + (0.6 x dense score). Dense retrieval gets slightly more weight as it understands meaning better.

Query	Baseline (s)	Hybrid (s)
What is the largest rodent	0.019	0.021
How do beavers build dams	0.017	0.021
Where do chinchillas come from	0.019	0.020
nutria invasive species	0.015	0.021
squirrel food storage winter	0.026	0.020
Average	0.019	0.021

Only 0.002 second overhead - completely imperceptible to users. Helps most for keyword-heavy queries like "nutria coypu invasive species". Adds less value for purely semantic queries.

7.2 Cross-Encoder Reranking

Bi-encoder retrieval embeds query and chunks separately. Cross-encoder reads query and chunk together, scoring relevance more accurately. Process: retrieve 20 candidates with hybrid search, then score each with cross-encoder, return top 5.

Example - "How much does a capybara weigh": Cross-encoder ranked Rodent article chunk mentioning "66 kg" as most relevant (score: 9.89), demoting Capybara social behavior chunks. Shows reranking understands what chunk actually answers the question.

Method	Avg Latency
Baseline	0.011s
Hybrid	0.012s
Reranked	0.062s

Adds 0.051s overhead - still under 100ms and imperceptible given LLM takes several seconds.

7.3 Query Transformation

Three variations implemented using Qwen2 as the transformation model:

- Rewriting: Fixes vague queries with pronouns. "where do they live" becomes "Where do rodents live and what is their habitat?"
- Expansion: Adds related terms. "rodent teeth" becomes "rodent teeth incisors gnawing dental formula continuously growing"
- HyDE: Generates hypothetical answer and searches with that instead of the question. Better semantic match with stored chunks.

Main tradeoff: Each transformation requires a full LLM inference pass (~2-3 seconds). Most expensive technique but provides the most dramatic improvement for vague queries.

8. Phase 8 - Evaluation Framework

8.1 Metrics Explained

Metric	Definition	Perfect Score
Precision@k	Of top k chunks retrieved, how many are relevant?	1.00
Recall@k	Of all relevant docs, how many found in top k?	1.00
MRR	How high up is the first relevant chunk?	1.00
Correctness	Is the answer factually correct? (LLM judge)	3/3
Groundedness	Is the answer based on context, not hallucination?	3/3
Refusal	Did model correctly refuse unanswerable questions?	3/3

8.2 Baseline Pipeline Results

Question	P@3	R@3	MRR	Correct
What is the largest rodent?	1.00	1.00	1.00	3/3
How much does capybara weigh?	1.00	0.50	1.00	3/3
Where do chinchillas come from?	1.00	1.00	1.00	3/3
What are beavers known for?	1.00	1.00	1.00	3/3
Are rabbits rodents?	1.00	1.00	1.00	3/3
What do squirrels do in winter?	1.00	1.00	1.00	3/3
What is a nutria?	1.00	1.00	1.00	2/3
How long do guinea pigs live?	1.00	1.00	1.00	2/3
What do marmots do in winter?	1.00	1.00	0.50	2/3
What makes rodents different?	0.67	1.00	1.00	0/3
Where do pikas live?	1.00	1.00	1.00	3/3
Rat intelligence?	1.00	1.00	1.00	3/3
Chinchilla fur?	1.00	1.00	1.00	3/3
Capybara social behavior?	1.00	1.00	1.00	3/3
Second largest rodent?	0.67	0.50	0.33	2/3
Population of Brazil? (unansw.)	0.00	0.00	0.00	0/3
Who invented telephone? (unansw.)	0.00	0.00	0.00	2/3
Speed of light? (unansw.)	0.00	0.00	0.00	0/3
How to make carbonara? (unansw.)	0.00	0.00	0.00	0/3
Capital of Japan? (unansw.)	0.00	0.00	0.00	0/3

Key observations: Retrieval is excellent for answerable questions (most P@3=1.00, MRR=1.00). Unanswerable questions correctly return P@3=0.00. Minor issue: telephone question got Correctness=2/3 suggesting partial hallucination. The "rodents different" question got 0/3 correctness despite correct retrieval suggesting LLM judge scoring issue.

9. Phase 9 - Web UI

9.1 Gradio Interface

Built using Gradio with share=True for Colab tunneling, producing a public URL valid for the duration of the Colab session.

Tab 1 - Ask a Question:

- Text input for user question
- Checkboxes to toggle: Hybrid Search, Cross-Encoder Reranking, Query Rewriting
- Answer output field
- Confidence indicator: High/Medium/Low with numeric score
- Source citations with relevance scores and Wikipedia URLs
- Example questions for quick testing

Tab 2 - System Info: Model details, configuration, knowledge base statistics.

9.2 Launching

Run demo.launch(share=True) to get a public URL in format: https://abc123.gradio.live. URL is valid for the duration of the Colab session.

10. Testing Guide

10.1 Answerable Questions

Question	Expected Answer
What is the largest rodent in the world?	Capybara, weighs up to 66kg
How do beavers build their dams?	Using sticks, mud, branches near water
Where do chinchillas come from?	Andes mountains, South America
Are rabbits considered rodents?	No, they are lagomorphs
What do marmots do in winter?	They hibernate
What is special about chinchilla fur?	Extremely dense and soft, valuable in fur trade
What is the second largest rodent?	The beaver

10.2 Unanswerable Questions

All should return: "I cannot find this information in the available documents"

- What is the population of Tokyo?
- Who invented the telephone?
- How do you make pasta carbonara?
- What is the speed of light?

10.3 Testing Advanced Techniques

Technique	Test Query	Expected Improvement
Hybrid Search	nutria coypu invasive species	BM25 boosts exact keyword matches
Reranking	How much does a capybara weigh exactly?	66kg chunk ranked first
Query Rewriting	where do they live and what do they eat	Pronouns replaced, better results
Confidence Low	What is the GDP of France?	Low confidence, refusal returned
Confidence High	What is the capybara?	High confidence, answer generated

11. Known Limitations

Small model hallucination

Qwen2-1.5B occasionally provides answers even when it should refuse, as seen with the telephone question receiving Correctness=2/3 instead of correctly refusing.

Retrieval boundary cases

Multi-document questions like "What is the second largest rodent?" require combining information from two documents, which reduces precision and recall scores.

ChromaDB compatibility

ChromaDB has version conflicts with Colab pre-installed NumPy 2.0. Solution was to use FAISS as primary vector store.

Session dependency

All models and indexes must be reloaded at the start of each Colab session. FAISS index is saved to Google Drive and reloaded automatically.

Generation speed

LLM generation takes 15-20 seconds per answer on free tier T4 GPU. Acceptable for demonstration but would need optimization for production.

12. Setup & File Structure

12.1 Requirements

- transformers>=4.46
- torch>=2.10
- sentence-transformers>=5.3
- faiss-cpu
- rank-bm25
- gradio>=5.0
- accelerate
- bitsandbytes
- langchain-text-splitters
- wikipedia-api
- pandas
- nltk

12.2 Running the Project

- 1. Open notebook in Google Colab
- 2. Set runtime to GPU (Runtime > Change runtime type > T4 GPU)
- 3. Mount Google Drive when prompted
- 4. Run all cells in order from Phase 1 to Phase 9
- 5. After Phase 9 Cell 6 a public Gradio URL will be generated
- 6. Open the URL to access the web interface

12.3 Google Drive File Structure

- MyDrive/
- rag_project/
- documents/documents.json
- chunks/recursive_chunks_512.json (main pipeline)
- chunks/fixed_chunks_256/512/1000.json
- chunks/semantic_chunks_0.2/0.3/0.5.json
- vector_stores/faiss_index.bin (main vector store)
- vector_stores/faiss_chunks.json
- eval_results/eval_baseline.json
- eval_results/eval_hybrid.json
- eval_results/eval_reranked.json
- eval_results/eval_queryrewrite.json
- eval_results/eval_fulloptimized.json
- eval_results/evaluation_comparison.csv
- eval_results/similarity_matrices.png